

Efficiency–Accuracy Tradeoffs in CNNs via Pruning and Quantization on CIFAR-10

Kobe Kodachi, Ian Hsiao

Abstract

Convolutional neural networks (CNNs) achieve state-of-the-art performance in image classification tasks, but impose significant memory, computation, and energy costs that hinder their deployment and scalability. In this work, we investigate the accuracy-efficiency tradeoffs of model compression techniques on three widely used CNN architectures: AlexNet, SqueezeNet, and ResNet32. First, each model is trained on the CIFAR-10 dataset for 100 epochs using full-precision (FP32) arithmetic. Then we apply magnitude-based unstructured pruning at sparsity levels ranging from 10% to 70%, followed by quantization-aware training (QAT) on the best-performing pruned configurations. Using TensorRT-based benchmarking, we evaluate test accuracy, model size, throughput, latency, and estimated inference energy consumption across FP32, FP16, and INT8 precisions. Our results show that substantial reductions in model size and energy consumption can be achieved with minimal or no loss in accuracy. In particular, INT8 quantization yields up to $25\times$ energy savings, while pruning acts as an effective regularizer in overparameterized models. These findings demonstrate that joint pruning and quantization enable efficient deployment of CNNs while preserving performance.

Introduction

Convolutional neural networks (CNNs) deliver state-of-the-art accuracy in image classification but require substantial memory, computation, and energy, which limits their deployment on resource-constrained hardware and scalable systems. As models grow larger and more overparameterized, it becomes increasingly important to identify and remove redundancy without sacrificing performance. Model compression techniques such as pruning and quantization offer promising solutions, but their effectiveness depends strongly on network architecture and training strategy. In this work, we study magnitude-based unstructured pruning and quantization-aware training on AlexNet, SqueezeNet, and ResNet32 using the CIFAR-10 dataset, evaluating accuracy, model size, throughput, latency, and energy consumption to characterize architecture-dependent efficiency–accuracy tradeoffs and enable more efficient CNN deployment.

Copyright © 2024, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

Background and Related Work

Convolutional Network Architectures

Convolutional neural networks (CNNs) have become the standard for image classification due to their ability to hierarchically extract spatial features from visual data through the reuse of kernels. Early work in classification, such as AlexNet, revealed that deep convolutional networks could significantly boost accuracy on large-scale image datasets by utilizing more layers and parameters (Krizhevsky, Sutskever, and Hinton 2012). The large fully connected layers of AlexNet demonstrate a tradeoff between a model’s memory footprint and accuracy. Future work would go on to optimize this.

One of those works was SqueezeNet, which was designed to achieve AlexNet-level accuracy with drastically reduced parameters (Iandola et al. 2016). The authors replaced the large convolutional layers of AlexNet with “fire modules,” which consist of squeeze and expand layers. The squeeze layer acts as a bottleneck to reduce the number of input channels, thus reducing the size of subsequent layers. The expand layers use 1×1 and 3×3 convolutions on the squeeze output and then concatenate along the channel dimension. This pattern of squeezing and expanding allows for a balance between accuracy and efficiency.

ResNet introduced residual connections to address the problem of vanishing gradients during the training of deep neural networks (He et al. 2016). The skip connections improve stability during training and facilitate feature reuse, making the architecture more robust to changes and parameter reduction. ResNet32 is a version of this architecture with 32 layers designed for CIFAR-10 (32×32 inputs), compared to the previously mentioned models, which were trained on the large-scale ImageNet dataset (224×224). We chose to examine model compression techniques on these three models due to their proven effectiveness and varying sizes.

Model Compression via Pruning

Model pruning is a technique that aims to remove redundant or insignificant parameters from neural networks in order to reduce model size and computation cost. Han et al. demonstrated that large CNNs can be pruned without significant accuracy loss by removing weights with small magnitudes and retraining the network. This forms the basis of the “Deep

Compression” pipeline (Han, Mao, and Dally 2016), which exploits the existence of substantial redundancy in overparameterized networks.

Pruning methods are broadly categorized into structured and unstructured techniques. Unstructured pruning removes individual weights, which can significantly reduce parameter counts with irregular sparsity patterns. Irregular sparsity is when the sparsity is spread out unevenly, rather than entire layers being removed, which makes it hard to exploit in hardware. Structured pruning removes entire channels or filters, improving hardware efficiency at the cost of control over accuracy degradation (Li et al. 2017). In this work, we focus on magnitude-based unstructured pruning, utilizing quantization to leverage hardware speedup.

The Lottery Ticket Hypothesis further suggests that there are sparse subnetworks within dense networks capable of achieving comparable accuracy when trained in isolation (Frankle and Carbin 2019). As such, magnitude-based unstructured pruning was the best choice for this work, as both AlexNet and SqueezeNet were designed for larger data.

Quantization and Low Precision Inference

Quantization reduces the numerical precision of weights and activations, which decreases memory usage and accelerates inference. Post-training quantization reduces FP32 parameters to lower-precision, often resulting in accuracy degradation from the resulting noise (Jacob et al. 2018). Quantization-aware training (QAT) mitigates this effect by simulating quantization during training, allowing the model to adapt to reduced precision before being converted.

Prior research has shown that a variety of neural network architectures can be quantized to FP16 or INT8 with minimal accuracy loss through QAT (Jacob et al. 2018). However, the sensitivity to quantization varies across architectures and can have varying interactions with other techniques such as pruning.

Hardware and Energy Considerations

Aside from accuracy and size, energy efficiency has become a significant concern in the modern world. Companies invest billions of dollars to build and power data centers with thousands of GPUs. Techniques that can optimize a model’s power consumption are vital to the future of machine learning. The previously mentioned techniques of pruning and quantization can result in extreme energy savings in addition to their other benefits. Studies have shown that memory accesses often dominate energy consumption compared to arithmetic operations, particularly in deep CNNs (Horowitz 2014). As such, pruning and quantization present an opportunity to reduce a model’s memory footprint, enabling larger batch processing to get lower latency, higher throughput, and better energy efficiency.

Methodology/Approach

Model Modifications

To study architecture-dependent efficiency–accuracy trade-offs, we evaluate AlexNet, SqueezeNet, and ResNet32 on the CIFAR-10 dataset. ResNet32 was designed specifically

for CIFAR-10, so no structural modifications were necessary. AlexNet and SqueezeNet were designed for larger images, so we modified them to accommodate CIFAR-10’s smaller size. In addition to scaling input and kernel sizes down, we used Batch Normalization, dropout, and data augmentation to improve generalization and mitigate overfitting.

Training Setup

All models were trained for 100 epochs using the following configuration to obtain the FP32 baseline:

- **Loss function:** Cross-Entropy Loss

$$\mathcal{L} = - \sum_{i=1}^C y_i \log(\hat{y}_i) \quad (1)$$

- **Optimizer:** Adam with learning rate $lr = 10^{-3}$ and weight decay 10^{-4}
- **Batch size:** 128 samples
- **Device:** NVIDIA A100

Pruning Strategy

We apply magnitude-based unstructured pruning to evaluate sparsity tolerance:

$$W' = W \odot \mathbf{1}_{|W| > \tau} \quad (2)$$

We used pruning levels: 10%, 30%, 50%, and 70% of total parameters. After pruning, each model was fine-tuned for 10 additional epochs to recover accuracy lost due to weight removal. This iterative pruning-finetuning procedure allows us to measure accuracy degradation versus sparsity.

Quantization Aware Training

After pruning, we took the pruned model with the highest accuracy and applied QAT, targeting INT8 precision. We simulate low-precision weights and activations during forward propagation and compute gradients in full precision to get accurate updates. After QAT the learned scale and zero-point parameters allow for minimal accuracy loss upon conversion. After conversion we expect that the compressed models retain comparable performance to their FP32 counterparts while reducing model size and energy consumption.

Benchmarking

For our benchmarking, we evaluated accuracy, latency, and throughput across multiple checkpoints of each model. Benchmarks were collected using TensorRT, NVIDIA’s high-performance inference optimization framework for GPUs built on CUDA. TensorRT accepts ONNX-formatted models as input and applies a range of optimizations, including operator fusion, kernel autotuning, and memory layout optimization, to generate inference engines optimized for the target device. In our workflow, ONNX acts as the intermediary between the PyTorch-trained models and TensorRT. For this work, we exported our trained `.pth` files to ONNX and used TensorRT to build inference engines for each model using the CIFAR-10 test set and comparing predicted labels with the ground truth.

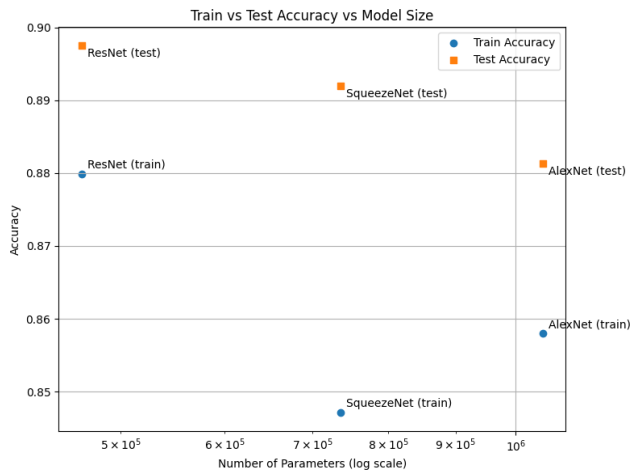


Figure 1: Graph of Train vs Test Accuracy vs Model Size

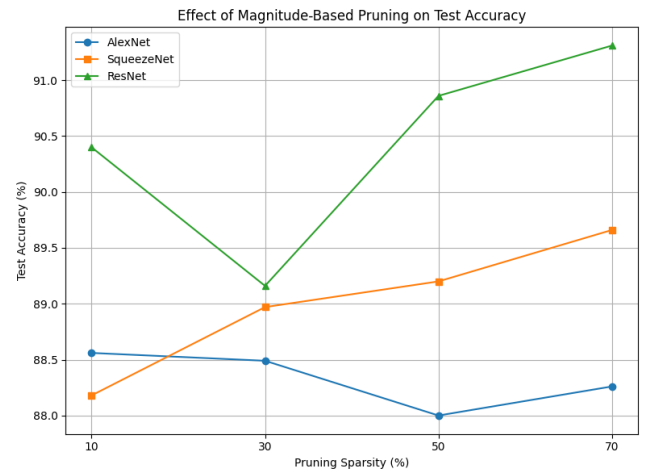


Figure 2: Graph of Accuracy vs Sparsity

Evaluation and Results

Evaluation Metrics

Model performance is evaluated using the following metrics:

- **Test Accuracy (%)**: Classification accuracy on the CIFAR-10 test set.
- **Sparsity (%)**: Fraction of pruned weights relative to the original dense model.
- **Engine Size (MB)**: Total parameter count and memory footprint.
- **Energy Efficiency**: Estimated inference energy in mJ derived from operation counts and precision scaling, following established energy models (Horowitz 2014).

These metrics capture both predictive performance and deployment efficiency, enabling fair comparison across architectures and compression levels.

Baseline Performance

All models are first evaluated in their FP32 configurations after 100 training epochs. These baseline models serve as reference points for evaluating the impacts of pruning and quantization. Figure 1 shows that ResNet32 achieves the highest baseline accuracy of 89.74%, followed by SqueezeNet at 89.19% and AlexNet at 88.13%. This ordering reflects the architectural advantages of residual connections and modern design principles.

Despite the larger size of SqueezeNet and AlexNet, we can see that they offer no advantage when working with a smaller dataset such as CIFAR-10. The comparable accuracy of these models dictates that size alone does not determine performance, and that architectural design plays a role in parameter efficiency.

Pruning Results

AlexNet exhibits the most stable accuracy across each pruning level, remaining around 88%. This suggests significant parameter redundancy, particularly within its fully con-

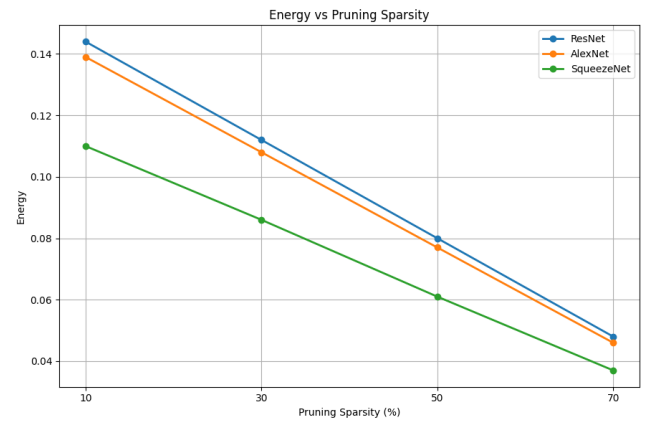


Figure 3: Graph of Energy vs Pruning Sparsity

nected layers. This aligns with expectations, as AlexNet was originally designed for larger images.

SqueezeNet shows an increase in accuracy as sparsity increases, with test accuracy improving from 88.18% at 10% sparsity to 89.66% at 70% sparsity. This increase in test accuracy suggests that pruning acts as a form of regularization to reduce overfitting. Like AlexNet, SqueezeNet was also designed for large images, so redundant parameters were expected.

ResNet shows the strongest response to pruning. While a temporary accuracy dip occurs at 30% sparsity (89.16%), performance recovers and surpasses lower sparsity levels at higher pruning rates, reaching 91.31% accuracy at 70% sparsity. This trend is likely a combination of the residual connections and smaller network size. The fewer parameters mean that each parameter contributes more, so we expect more noise across pruning levels. The overall increase in accuracy can be attributed to the residual connections supporting better gradient flow.

These results, seen in Figure 2, support the view that pruning can act as a regularizer in overparameterized models.

Model	Test Accuracy (%)	Engine Size (MB)	Throughput (batch 128)	Latency (s)	Energy (J)
ResNet FP32	89.74	2.1	176.3	0.726	0.123
ResNet FP16	89.74	1.2	294.25	0.435	0.029
ResNet INT8	89.74	0.795	306.95	0.417	0.005
AlexNet FP32	88.13	4.3	390.24	0.328	0.155
AlexNet FP16	88.14	2.2	731.43	0.175	0.037
AlexNet INT8	88.16	1.2	1057.85	0.121	0.007
SqueezeNet FP32	89.19	3.3	173.68	0.737	0.123
SqueezeNet FP16	89.21	1.9	312.2	0.41	0.029
SqueezeNet INT8	89.18	1.3	356.55	0.359	0.005

Figure 4: Performance Metrics of Quantized Models

Across all three models, magnitude-based pruning did not result in significant accuracy loss, even at high sparsity levels. This indicates that the baseline models were substantially overparameterized. However, we can see that the response to pruning varies by architecture, as AlexNet shows the least sensitivity to pruning, further solidifying our initial assumption that many parameters contribute little to final performance. In contrast, the graph shows that SqueezeNet and ResNet did benefit from pruning, with better accuracy at higher sparsity levels.

Furthermore, using the energy estimation models defined by Horowitz et al., we can see a significant reduction in energy by reducing the effective number of MACs (Horowitz 2014). Effective MACs were calculated using the pruning percent multiplied by baseline MACs. Figure 3 shows that even with FP32, a significant reduction in energy consumption is observed. Together with the previous graph, this illustrates that over parameterized models can be made more hardware-efficient through pruning. However, pruning alone only reduces the amount of computation performed, while the cost of each operation remains unchanged.

Quantization Results

Figure 4 summarizes the performance, memory footprint, throughput, latency, and total energy consumption of ResNet, AlexNet, and SqueezeNet under FP32, FP16, and INT8 precision. Across all models, reducing numerical precision yields substantial efficiency gains with negligible impact on test accuracy. ResNet maintains a consistent accuracy of 89.74% across all precisions, while achieving a 63% reduction in model size from FP32 (2.1 MB) to INT8 (0.795 MB), a 1.46 $\times$ increase in throughput, and a nearly 25 $\times$ reduction in total energy consumption (0.123 J $\rightarrow$ 0.005 J). SqueezeNet exhibits similar trends, with test accuracy remaining around 89.2% and total energy dropping from 0.123 J (FP32) to 0.005 J (INT8), while memory is reduced by more than 60%. AlexNet achieves a large throughput improvement and sees energy consumption reduced from 0.155 J to 0.007 J.

These results demonstrate that INT8 quantization pro-

vides dramatic reductions in memory and energy while preserving accuracy across these architectures. Additionally, FP16 offers a favorable tradeoff between model size and latency as it enables over 2 $\times$ faster inference compared to FP32 with minimal accuracy loss. Overall, these findings highlight the effectiveness of low-precision inference in enabling the high-throughput and energy-efficient deployment of CNNs.

Combining Pruning and Quantization

Lastly we analyzed the performance of the highest-accuracy pruned and INT8-quantized models for ResNet, AlexNet, and SqueezeNet. All models achieve significant energy savings with minimal loss in accuracy. ResNet (70% pruned) reaches 88.67% accuracy, 0.002 J energy, 210 images/sec throughput, and 0.609 s latency. SqueezeNet (70% pruned) gets 88.42% accuracy, 0.002 J energy, 220.69 images/sec throughput, and 0.58 s latency. AlexNet (10% pruned) achieves 86.84% accuracy, 0.002 J energy, 526.74 images/sec throughput, and 0.243 s latency. These results demonstrate that the combination of pruning and quantization can greatly reduce energy use while maintaining accuracy.

Overall, the performance will still vary by architecture, as ResNet and SqueezeNet tolerate high sparsity, whereas AlexNet requires lighter pruning.

Conclusion

In this work, we studied the efficiency-accuracy tradeoffs of model compression techniques on AlexNet, SqueezeNet, and ResNet32 trained on the CIFAR-10 dataset. Through magnitude-based unstructured pruning and quantization-aware training, we demonstrated that significant reductions in model size, inference energy, and latency can be achieved with minimal impact on test accuracy. Our results show that pruning can act as an effective regularizer in overparameterized models, particularly for architectures originally designed for larger input resolutions. Quantization further amplifies these benefits by reducing the cost of arithmetic operations, with INT8 precision providing the largest gains in energy efficiency and throughput. The effectiveness of compression techniques varies across architectures, with ResNet32 and SqueezeNet tolerating higher sparsity levels than AlexNet due to residual connections and parameter-efficient designs respectively. Overall, these findings highlight the importance of jointly considering model architecture, pruning strategy, and numerical precision when deploying CNNs on resource-constrained hardware. The biggest shortcoming of this work is that we were unable to work with most datasets due to time and resource constraints. As such, future work may investigate whether these techniques are equally effective when applied to larger data sizes and models.

Acknowledgments

Code is available at <https://github.com/kkodachi/CNN-Benchmarks.git>.

References

- 335 Frankle, J.; and Carbin, M. 2019. The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks. In *International Conference on Learning Representations (ICLR)*.
- 340 Han, S.; Mao, H.; and Dally, W. J. 2016. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. In *International Conference on Learning Representations (ICLR)*.
- 345 He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Horowitz, M. 2014. 1.1 Computing's Energy Problem (and What We Can Do About It). *ISSCC*.
- 350 Iandola, F. N.; Han, S.; Moskewicz, M. W.; Ashraf, K.; Dally, W. J.; and Keutzer, K. 2016. SqueezeNet: AlexNet-level Accuracy with 50x Fewer Parameters and 0.5MB Model Size. *arXiv preprint arXiv:1602.07360*.
- 355 Jacob, B.; Kligys, S.; Chen, B.; Zhu, M.; Tang, M.; Howard, A.; Adam, H.; and Kalenichenko, D. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- 360 Krizhevsky, A.; Sutskever, I.; and Hinton, G. E. 2012. ImageNet Classification with Deep Convolutional Neural Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- 365 Li, H.; Kadav, A.; Durdanovic, I.; Samet, H.; and Graf, H. P. 2017. Pruning Filters for Efficient ConvNets. In *International Conference on Learning Representations (ICLR)*.